

Curriculum Intelligence Engine — Week 1: Project Proposal v2

Group 9 · Roll Nos: J048, J049, J050, J052, J053 · AI University NLP Course Project 2026 · revised 9 Oct 2026 (v1: 11 Sep)

CODE

```
Repository : https://github.com/jainamsethia/Curriculum-Intelligence-Engine
Tag        : week-1-redo
Commit     : 6e80a22cf7b428ba0ba6b006b45cde2e84057c93
Run        : python -m curriculum_engine.run all      (NMIMS_ZIP=<path to "B TECH.zip">; see README)
Folders    :
.gitignore
README.md
curriculum_engine/  the NLP engine (parser, data, overlap, redundancy, prerequisites, diff, Bloom, evaluation)
labels/            LLM-annotated reference labels (ids + labels per pass + consensus)
legacy/           old Week 3-4 scripts and reports (history only)
legacy_mit/       old MIT OCW scripts (not used)
reports/          report templates (numbers are placeholders)
requirements.txt
results/          metrics JSON + figures per week
tests/            unit tests (pytest)
tools/            report builder, report checker, submission packer
```

Feedback addressed

Instructor comment	What we changed	Evidence
Week 2: "use the course catalog from your own college ... value to NMIMS, and not for MIT"	NMIMS B.Tech semester syllabus booklets are the only dataset for every result. The MIT collector is archived and not used.	curriculum_engine/parse.py, results/dataset/parse_stats.json, legacy_mit/
Week 3: "Progress relative to previous week is minimal"	Every capability now has a measurable target (table below); every weekly report opens with a progress table against the previous week.	results/plan/targets.json
Week 4: "Where is the code? ... overlapping content, missing pre-requisites, detect redundancies"	Overlap, redundancy and missing prerequisites are capabilities 1–3 and the headline metrics of every week. Every report starts with this CODE block and ships with a code zip.	this section; reports/templates/
Our own audit: the graded Week 3 used MIT pages while Week 4 cited NMIMS data	One dataset story from Week 1 on; all numbers are regenerated, none are copied from old reports.	tools/check_reports.py

Progress vs previous version

Aspect	Proposal v1 (11 Sep)	Proposal v2 (this report)
Dataset	public syllabi first, NMIMS optional, 40–60 documents	NMIMS semester syllabus booklets only: 2,366 course versions of 655 courses
Success criteria	none	a numeric target per capability, fixed before any result
Compare curricula	course pairs only	course pairs and whole programmes (programme vs programme, year vs year)
Emerging topics, ontology	named, no method	cited reference list + embedding coverage; topic clusters with reviewed labels
Evaluation labels	"small human-annotated set"	LLM-annotated reference labels with a documented multi-pass protocol, plus label-free tests

Problem and users

Universities revise curricula periodically, but comparing course descriptions, syllabi and learning outcomes across semesters, programmes and years is done by hand. It is slow and inconsistent, and makes it hard to spot overlapping topics, redundant content, missing prerequisites or gaps in emerging topics. Users: Board of Studies and curriculum committees, heads of department checking electives for redundancy, the academic office mapping outcomes to Bloom's taxonomy, and faculty designing a new course.

Input and output

Input: two courses, two programmes or two academic years of a programme, given as NMIMS syllabus PDF/DOCX/text or as course ids from the parsed dataset. Output: a JSON report with confidence scores, for example:

```
{
  "course_a": "...",
  "course_b": "...",
  "overlap": {
    "percent": "72%",
    "confidence": 0.86,
    "redundancy_warning": false
  },
  "common_topics": ["Probability", "Regression", "Classification"],
  "unique_to_a": ["Time Series"],
  "unique_to_b": ["Transformers", "Large Language Models"],
  "missing_prerequisites": [
    {
      "course": "...",
      "assumes": "...",
      "reason": "not taught in an earlier semester of the programme"
    }
  ],
  "outcome_bloom": [
    {
      "outcome": "...",
      "level": "Apply",
      "confidence": 0.7
    }
  ]
}
```

Dataset: NMIMS semester syllabus booklets

- Source: the B.Tech syllabus archive of our college (2,413 non-exam PDFs across all years and branches, organised by year of study, programme and semester). Past exam papers in the archive are excluded: they are not syllabi.
- Coverage limitation: 531 of 2,413 PDFs have a text layer and are parsed; 1,882 are scanned images. We have no OCR engine, so most pre-2020 booklets and some programme-year booklets are missing. OCR is future work.
- Parsed: 2,366 course versions of 655 distinct courses (472 with two or more versions), academic years 2020-21 to 2026-27. Each record holds programme, semester, academic year, credits, prerequisites, objectives, 8,650 learning outcomes (546 carry the faculty's own K/L Bloom tag) and 14,747 units.
- Versions vs courses: all versions of a course form one family (same code or same name). Comparing two versions is a separate version-change task, and a family is never split between training and test data.
- Data governance: no student data; the parsed fields contain no faculty names or e-mails (checked); grading and attendance tables are cut. The parsed text stays out of the public repository; only code, ids, labels and metrics are published.

Capabilities, methods and success criteria

#	Capability	Baseline → improved	Ground truth	Metric: target
1	Course-pair overlap: score, common and unique topics	TF-IDF cosine → chunk-level embeddings (MaxSim) + topic labels	LLM-annotated reference labels (0/1/2); version pairs reported separately	Spearman ≥ 0.600 , ROC-AUC ≥ 0.850 , F1 ≥ 0.700
2	Redundant content within a programme	course-level TF-IDF threshold → unit-level near-duplicates	reference labels on top flags; injected duplicate units	precision@20 ≥ 0.700 , injected recall@20 ≥ 0.800
3	Missing prerequisites	regex + exact course-name match → embedding resolution + programme graph (prerequisite not offered earlier; assumed topic not taught earlier)	removal of a prerequisite course; renamed-course test; reference labels on flags	removal recall ≥ 0.800 , flag precision ≥ 0.600 , false alarms on renamed courses ≤ 0.200
4	Compare two curricula (programme vs programme, year vs year)	course-name set difference → family + content matching, structured diff	known year-over-year changes	precision ≥ 0.850 , recall ≥ 0.850
5	Outcome → Bloom level (stretch)	verb lexicon → verb lexicon + embedding classifier / LLM	faculty K/L tags printed in the syllabi	accuracy ≥ 0.700 , within ± 1 level ≥ 0.850
6	Emerging-topic gaps	cited reference list vs programme coverage (embeddings)	reference labels on flagged gaps	precision ≥ 0.700
7	Topic model / ontology	unit-embedding clusters with class-TF-IDF names, grouped under a curated hierarchy	review of topic labels	labels accepted ≥ 0.700

NLP components used: semantic similarity and embeddings (overlap, redundancy, curriculum diff, emerging topics), information extraction (prerequisites, units, outcomes), classification (Bloom), topic modelling and ontology construction (topic labels, topic hierarchy). Emerging topics: a list of current topics (for example LLMs, retrieval-augmented generation, MLOps, AI safety, quantum computing, edge AI), each tied to a source we can verify; unverifiable entries are marked "to confirm". A programme has a gap when no unit of its courses is semantically close to a topic.

Labels and evaluation protocol

- Split by course family: 60% train, 20% validation, 20% test (seed 42). Thresholds and hyper-parameters are chosen on validation only; the test split is used once per reported run. 95% confidence intervals resample whole courses.
- LLM-annotated reference labels: stratified course pairs and detector flags are labelled in at least two independent passes with different prompt wordings (pass 1: a Claude model in a fresh context; pass 2: Phi-3 running locally), without any model scores. Disagreements go to a third pass; items without a 2-of-3 majority are marked uncertain and excluded. Agreement is reported as LLM-vs-LLM agreement. No human annotators were used; results are indicative. A human spot-check sheet (ten pairs per member) is provided for optional later checking.
- Label-free tests run alongside: faculty Bloom tags, injected overlap and duplicate units, removal of prerequisite courses, renamed courses, known year-over-year changes.

Architecture and API

Parser (PDF/DOCX → course records) → course families, programme structure (programme × semester × year) → embeddings → overlap, redundancy, prerequisite, diff, Bloom and emerging-topic modules → JSON. Reusable asset (Week 6): Python package `curriculum_engine` and a FastAPI service with POST `/compare-curricula`, POST `/map-outcomes`, POST `/find-overlaps`, POST `/check-prerequisites`, POST `/detect-redundancy`, POST `/emerging-topics` and GET `/health`, each returning confidence scores and model/version information. Links to the AI University: the Knowledge Assistant (Group 4) can answer "which courses overlap with X?" from our API; Communications Intelligence (Group 11) can draft circulars about curriculum changes from our diff reports.

Eight-week plan

Week (due)	Deliverable	Evidence of progress
Week 1 (11 Sep, v2 9 Oct)	this proposal	targets fixed
Week 2 (26 Sep)	NMIMS dataset, schema, families, programme structure, split, reference labels, baselines for capabilities 1–5	baseline metrics on the test split
Week 3 (26 Sep)	core model: MaxSim overlap, unit-level redundancy, programme-graph prerequisite checker, topic labels	same split and metrics vs Week 2
Week 4 (2 Oct)	ablations and improvements with confidence intervals	baseline-vs-improved table
Week 5 (9 Oct)	evaluation and error analysis: per programme, thresholds, robustness, failure cases	at least ten documented failures
Week 6 (16 Oct)	reusable asset: package + FastAPI + schemas + confidence + docs	API tests pass
Week 7 (23 Oct)	committee UI, integration with Groups 4/11 (adapter + mock), responsible-AI analysis	cross-component demo
Week 8 (30 Oct)	final report, README, slides, demo, architecture diagram, tag v1.0	clean-clone run

Risks and mitigations

Risk	Mitigation
Scanned PDFs leave programme-year gaps, so a prerequisite can look "missing"	coverage-aware checks; coverage table per programme; OCR as future work
Reference labels come from LLMs, not people	multi-pass protocol, agreement reported, label-free tests, optional human spot check, stated in every report
Templated syllabi share wording, so lexical similarity is already strong	unit-level analysis, honest "no clear difference" verdicts
CPU-only machine	cached embeddings, small local LLM, LLM steps optional

Responsible AI

Scores support a committee; they never cut or merge courses automatically. Reports show the evidence (matched units, prerequisite text) behind each flag, include confidence, and are checked for faithfulness when an LLM writes text. Results are broken down by department to spot uneven quality. No personal data is processed.

AI assistance

Code, labels and report drafting were produced with AI assistance (Claude; Phi-3 locally for labelling and extraction). All numbers in this report are generated from `results/*.json` by `tools/build_report.py` and verified by `tools/check_reports.py`. LLM-annotated reference labels are used for evaluation; no human annotators were used; results are indicative.